Cloud security and IAM

Control who and what can do what in the cloud: identities, roles and policies on AWS and Azure, least privilege, secrets and keys, private networking, audit logs and debugging access errors.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. Identities, principals and permissions (11 min)
2. AWS IAM: policies, roles and assume-role (13 min)
3. Azure: Entra ID, RBAC and managed identities (12 min)
4. Least privilege and access reviews (12 min)
5. Secrets and encryption with KMS and key vaults (13 min)
6. Network boundaries and private access (12 min)
7. Logging, detection and audit trails (12 min)
8. Debugging access denied and cloud security incidents (14 min)
9. GCP IAM and service accounts (12 min)
10. Workload identity federation and OIDC trust (12 min)
11. Permission boundaries, SCPs and guardrails (13 min)
12. IAM policy analysis and unused access (12 min)
Cheat sheet

Lesson 1. Identities, principals and permissions

11 minutes. Learn the vocabulary of cloud access: who is asking, what they may do, on which resource, and how a decision is made.

Every cloud action is a request: someone or something asks to do an action on a resource. The cloud checks identity, then permissions, then conditions, and answers allow or deny. Almost every security problem in the cloud is a mistake in that chain: the wrong identity, too many permissions, or a condition nobody checked. This lesson gives you the vocabulary before the provider-specific lessons.

What you will learn

- Separate authentication from authorization
- Name the parts of an access decision
- Tell human identities from workload identities
- Spot the most common identity mistakes

Authentication and authorization

Authentication answers "who are you?" with a password, a hardware key, a token or a certificate. **Authorization** answers "what may you do?" by comparing the request with policies. A system can authenticate you correctly and still wrongly authorize you, and the second mistake is the common one. Multi-factor authentication protects the first step; least privilege protects the second.

The parts of a decision

Think of four fields. The **principal** is who asks (a person, a service, a pipeline). The **action** is what they want (read an object, start a server). The **resource** is the thing acted on (one bucket, one subscription). The **conditions** narrow it further (only from this network, only today, only with MFA). A policy is a statement that allows or denies a combination of these. Most clouds start from deny by default: with no matching allow, the answer is no. An explicit deny beats any allow.

Humans and workloads

People sign in through a central identity provider, ideally with single sign-on and MFA, and receive short-lived sessions. Workloads such as a virtual machine, a function or a Kubernetes pod need identities too. The safe pattern is an identity that the platform attaches to the workload, which gives it temporary credentials automatically. The unsafe pattern is a long-lived access key copied into a configuration file or a repository, where it is leaked, shared and never rotated.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Identities, principals and permissions fits together

Part	What it does	What goes wrong
Authentication and authorization	Authentication answers "who are you?" with a password, a hardware key, a token or a certificate.	Check the Common mistakes list below this diagram for what usually goes wrong here.
The parts of a decision	Think of four fields. The principal is who asks (a person, a service, a pipeline).	Check the Common mistakes list below this diagram for what usually goes wrong here.
Humans and workloads	People sign in through a central identity provider, ideally with single sign-on and MFA, and receive short-lived sessions.	Long-lived keys in code or files.

Flow: Authentication and authorization to The parts of a decision / The parts of a decision to Humans and workloads

MARGA by EknathaLabs marga.eknathalabs.com

Common mistakes

Using the root or owner account for daily work; creating one shared login for a team; giving administrator rights "just to get it working"; leaving keys with no expiry; and never removing access when someone changes team. Treat identities like inventory: know who has what, why, and when it ends.

Incident: The key in the repository

> **Incident:** *A test key for a deploy pipeline was committed to a repository and found by a scanner three days later.*

```
$ git log --oneline -1 -- deploy/.env
3f9a1c2 add deploy env
$ aws iam list-access-keys --user-name ci-deploy
AccessKeyId: AKIA...EXAMPLE  Status: Active  CreateDate: 2024-03-02
```

The key belonged to a user with broad permissions and no expiry. The team deactivated it, reviewed the audit log for use from unknown addresses, replaced it with a role that the pipeline assumes for short sessions, and added secret scanning before every commit lands.

Common mistakes

- Using the root account for daily work
- Sharing one login between people
- Long-lived keys in code or files
- Skipping MFA for administrators

Check yourself

What does authorization decide?

Answer: What an identity may do

What happens when no policy allows an action?

Answer: It is denied by default

Which is safer for a workload?

Answer: An identity the platform attaches with temporary credentials

What beats an allow?

Answer: An explicit deny

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 2. AWS IAM: policies, roles and assume-role

13 minutes. Read and write AWS IAM policies, use roles instead of users for workloads, and understand how sessions and boundaries combine.

AWS Identity and Access Management is the permission system for the whole account. You will meet users, groups, roles and policies. The most important idea is the role: an identity with no permanent password or key that a trusted principal can assume for a short session. If you remember one rule, make it this one: people and workloads should assume roles, not hold long-lived keys.

What you will learn

- Read a JSON policy
- Know the difference between identity-based and resource-based policies
- Use roles and assume-role for workloads and cross-account access
- Understand how boundaries and service control policies limit permissions

Reading a policy

A policy lists statements. Each has an effect (Allow or Deny), actions, resources and optional conditions.

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["s3:GetObject"],
    "Resource": "arn:aws:s3:::reports-prod/summaries/*"
  }]
}
```

This allows reading objects under one prefix of one bucket and nothing else. A wildcard such as `"Action": "*"` with `"Resource": "*"` is administrator access.

Two kinds of policy

An **identity-based policy** is attached to a user, group or role and says what that identity may do. A **resource-based policy** is attached to a resource, such as a bucket or a queue, and says who may use it, including principals from other accounts. A request is allowed when the right policies allow it and nothing denies it.

Roles and assume-role

A role has two parts: a **trust policy** saying who may assume it, and **permission policies** saying what it can do. A caller asks the Security Token Service to assume the role and receives temporary credentials, usually valid for an hour or less. Compute services attach roles directly: an instance profile for a virtual machine, an execution role for a function, a task role for a container. Cross-account access works the same way: the target account trusts a role in the source account.

Limits above policies

A **permissions boundary** caps what an identity can ever be granted. A **service control policy** in AWS Organizations caps what whole accounts can do, even for their administrators. They never grant by themselves; they only restrict. Use them for guardrails such as "no one may disable audit logging" or "only approved regions".

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How AWS IAM: policies, roles and assume-role fits together

Part	What it does	What goes wrong
Reading a policy	A policy lists statements. Each has an effect (Allow or Deny), actions, resources and optional conditions.	Using IAM users with keys for workloads.
Two kinds of policy	An identity-based policy is attached to a user, group or role and says what that identity may do.	Forgetting resource-based policies when debugging.
Roles and assume-role	A role has two parts: a trust policy saying who may assume it, and permission policies saying what it can do.	Trust policies that allow any principal.
Limits above policies	A permissions boundary caps what an identity can ever be granted.	They never grant by themselves; they only restrict.

Flow: Reading a policy to Two kinds of policy / Two kinds of policy to Roles and assume-role / Roles and assume-role to Limits above policies

Incident: The wildcard that opened the bucket

Incident: *A developer fixed an access error by granting s3:* on all buckets to a build role.*

```
$ aws iam get-role-policy --role-name build --policy-name tmp
"Action": "s3:*", "Resource": "*"
$ aws iam simulate-principal-policy --policy-source-arn arn:aws:iam::123456789012:role/build --act \
    ion-names s3:DeleteObject --resource-arns arn:aws:s3:::customer-exports/a.csv
EvalDecision: allowed
```

The simulator showed the build role could delete customer exports. The team replaced the wildcard with the one bucket and prefix the build needs, added a boundary that denies deletes on data buckets, and put policy checks into code review.

Common mistakes

- Granting "*" to make an error disappear
- Using IAM users with keys for workloads
- Trust policies that allow any principal
- Forgetting resource-based policies when debugging

Check yourself

What is a role?

Answer: An identity assumed for a short session without a permanent key

What does a trust policy define?

Answer: Who may assume the role

What do service control policies do?

Answer: Cap what accounts can be granted

Which action is administrator access in a policy?

Answer: Action *on Resource*

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 3. Azure: Entra ID, RBAC and managed identities

12 minutes. Understand how Azure separates the directory from resource permissions, assign roles at the right scope, and use managed identities.

Azure splits identity from access to resources. Microsoft Entra ID is the directory that holds users, groups and applications. Azure role-based access control decides what those identities can do to subscriptions, resource groups and resources. Learning to keep the two ideas apart makes most Azure access questions easy to answer.

What you will learn

- Explain Entra ID and Azure RBAC
- Choose the right scope for a role assignment
- Use managed identities instead of secrets
- Use conditional access and privileged access sensibly

Directory and resources

Entra ID authenticates people and applications and issues tokens. It also has directory roles, such as managing users, which are separate from Azure resource roles. Azure RBAC governs resources: a **role definition** lists allowed actions, a **scope** says where it applies, and a **role assignment** ties a principal to a role at a scope.

Scopes and inheritance

Scopes form a hierarchy: management group, subscription, resource group, resource. An assignment at a higher scope is inherited by everything below it. Assign roles at the lowest scope that works. Giving Contributor on a subscription to fix one resource group problem grants access to everything in it.

Built-in roles

Start with built-in roles such as Reader, Contributor and Owner, and narrower ones like Storage Blob Data Reader. Owner can also assign roles, so reserve it. Note that control-plane roles and data-plane roles differ: being Contributor on a storage account does not automatically allow reading blob data.

Managed identities

A **managed identity** is an identity that Azure creates for a resource, such as a virtual machine, app service or function, and manages for you. The application requests a token from a local endpoint and uses it to reach other services, with no secret to store or rotate. A system-assigned identity lives and dies with its resource; a user-assigned identity is created separately and can be shared.

Protecting sign-ins

Conditional access policies require MFA, compliant devices or trusted locations. Privileged Identity Management gives administrators just-in-time, time-limited elevation with approval and logs, instead of standing permanent rights.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Azure: Entra ID, RBAC and managed identities fits together

Part	What it does	What goes wrong
Directory and resources	Entra ID authenticates people and applications and issues tokens.	Confusing directory roles with resource roles.
Scopes and inheritance	Scopes form a hierarchy: management group, subscription, resource group, resource.	Assigning roles at the subscription level by default.
Built-in roles	Start with built-in roles such as Reader, Contributor and Owner, and narrower ones like Storage Blob Data Reader.	Leaving permanent Owner rights.
Managed identities	A managed identity is an identity that Azure creates for a resource, such as a virtual machine, app service or function, and manages for you.	Storing client secrets when a managed identity works.
Protecting sign-ins	Conditional access policies require MFA, compliant devices or trusted locations.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Directory and resources to Scopes and inheritance / Scopes and inheritance to Built-in roles / Built-in roles to Managed identities / Managed identities to Protecting sign-ins

Incident: Contributor at the subscription

> **Incident:** *A contractor needed to restart one web app and was made Contributor on the whole subscription.*

```
$ az role assignment list --assignee contractor@example.com --all -o table
Role         Scope
Contributor  /subscriptions/0000-1111
```

The contractor could change every resource, including production databases. The team removed the assignment, created a custom role allowing only restart on that app, assigned it at the resource scope, and enabled time-limited elevation for similar needs.

Common mistakes

- Assigning roles at the subscription level by default
- Confusing directory roles with resource roles
- Storing client secrets when a managed identity works
- Leaving permanent Owner rights

Check yourself

What does Entra ID provide?

Answer: Identities and sign-in for users and applications

What is a role assignment made of?

Answer: Principal, role definition and scope

Why use a managed identity?

Answer: No secret to store or rotate

Where should you assign a role?

Answer: At the lowest scope that works

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 4. Least privilege and access reviews

12 minutes. Shrink permissions to what is needed, with tools to find unused access, and keep it small with regular reviews.

Least privilege means every identity has only the access its job needs, for only as long as needed. Nobody gets it perfect on day one, and permissions grow over time. The practical method is to start narrow, measure what is really used, remove the rest, and review on a schedule so access does not quietly accumulate.

What you will learn

- Describe least privilege in practice
- Use access analysis tools to find unused permissions
- Design roles around jobs, not people
- Run a simple access review

Start narrow, then measure

Begin with the permissions you know are needed. When a request is denied, read the exact action and resource in the error and add only that. In the other direction, look at what existing identities actually use. AWS offers access analysis showing last-accessed services and can generate a policy from recorded activity; Azure and others show similar usage data. Remove what has not been used for a period such as 90 days, after checking with the owner.

Roles for jobs

Create roles for functions (read-only auditor, deployer for one application, database operator) and assign people to them through groups. This avoids a unique set of permissions per person that nobody can review. Separate duties: the person who approves a change should not be the only one who can make and deploy it.

Temporary and conditional access

Prefer time-limited access for rare tasks, with approval and automatic expiry. Add conditions: require MFA, restrict to expected networks or regions, require a ticket reference in the session name. Break-glass accounts for emergencies should exist, be stored safely, be monitored, and trigger an alert when used.

Access reviews

Every quarter, list who has privileged access and ask the owner to confirm. Check service accounts, third-party integrations and old contractors. Record the date, the reviewer and the changes made. A review that finds nothing to remove for a year probably is not looking at real data.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Least privilege and access reviews fits together

Part	What it does	What goes wrong
Start narrow, then measure	Begin with the permissions you know are needed.	Reviews without usage data.
Roles for jobs	Create roles for functions (read-only auditor, deployer for one application, database operator) and assign people to them through groups.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Temporary and conditional access	Prefer time-limited access for rare tasks, with approval and automatic expiry.	Granting broad rights because tuning takes time.
Access reviews	Every quarter, list who has privileged access and ask the owner to confirm.	No owner for service accounts.

Flow: Start narrow, then measure to Roles for jobs / Roles for jobs to Temporary and conditional access / Temporary and conditional access to Access reviews

Incident: Nobody remembers why

Incident: *A security review found 31 service accounts with administrator rights and no owner.*

```
$ aws iam generate-service-last-accessed-details --arn arn:aws:iam::123456789012:role/legacy-batch
$ aws iam get-service-last-accessed-details --job-id 1234
ServiceNamespace: s3  LastAuthenticated: 2023-11-02
ServiceNamespace: ec2  LastAuthenticated: -
```

The report showed the role touched only one service, long ago. The team assigned owners, disabled unused accounts, replaced administrator rights with specific policies, and scheduled a quarterly review with a record of each decision.

Common mistakes

- Granting broad rights because tuning takes time
- No owner for service accounts
- Reviews without usage data
- No emergency access plan

Check yourself

What is least privilege?

Answer: Only the access needed, for only as long as needed

What helps you find unused permissions?

Answer: Last-accessed and usage analysis

Why design roles for jobs?

Answer: They are reviewable and reusable

What should break-glass accounts do when used?

Answer: Raise an alert

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 5. Secrets and encryption with KMS and key vaults

13 minutes. Store secrets in a managed service, understand keys and envelope encryption, and rotate credentials without outages.

Passwords, tokens and keys must live somewhere. The wrong places are source code, container images, chat messages and shared documents. The right place is a managed secrets store, with access controlled by identity, every read logged, and rotation built in. Encryption keys are a related but different subject, and cloud key management services keep them safe so you never handle raw key material.

What you will learn

- Choose a place for secrets and keep them out of code
- Explain encryption at rest and in transit
- Understand keys, envelope encryption and key policies
- Rotate secrets safely

Secrets stores

AWS Secrets Manager, AWS Systems Manager Parameter Store and Azure Key Vault store secrets and return them to authorised identities at run time. The application fetches the value with its workload identity, holds it in memory and never writes it to disk or logs. Access is a permission like any other, so a pipeline may read only its own secrets. Every read appears in the audit log.

Keys and envelope encryption

A key management service (AWS KMS, Azure Key Vault keys, Google Cloud KMS) creates keys and performs encryption without exposing the key. For large data, **envelope encryption** is used: a data key encrypts the data, and the master key in KMS encrypts the data key. Storing the encrypted data key beside the data keeps performance high and control central. Key policies decide who can use and who can manage a key; separate those two powers.

At rest and in transit

Encrypt storage at rest by default for disks, databases and object storage, using managed or customer-managed keys when rules demand control. Encrypt traffic in transit with TLS and reject plain connections. Encryption does not replace access control: anyone with permission to decrypt sees the plain data.

Rotation

Rotate credentials on a schedule and immediately after exposure. Safe rotation supports two valid versions for a while: create the new, switch consumers, then retire the old. Managed rotation for database passwords can do this automatically. Prefer short-lived tokens, which make rotation a non-event.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Secrets and encryption with KMS and key vaults fits together

Part	What it does	What goes wrong
Secrets stores	AWS Secrets Manager, AWS Systems Manager Parameter Store and Azure Key Vault store secrets and return them to authorised identities at run time.	Secrets in code, images or logs.
Keys and envelope encryption	A key management service (AWS KMS, Azure Key Vault keys, Google Cloud KMS) creates keys and performs encryption without exposing the key.	Keys and envelope encryption A key management service (AWS KMS, Azure Key Vault keys, Google Cloud KMS) creates keys and performs...
At rest and in transit	Encrypt storage at rest by default for disks, databases and object storage, using managed or customer-managed keys when rules demand control.	Giving everyone permission to decrypt.
Rotation	Rotate credentials on a schedule and immediately after exposure.	No rotation plan.

Flow: Secrets stores to Keys and envelope encryption / Keys and envelope encryption to At rest and in transit / At rest and in transit to Rotation

Incident: The environment variable in the image

> **Incident:** *A container image published to a registry contained a database password in an environment variable.*

```
$ docker history --no-trunc shop-api:2.3 | grep DB_PASSWORD
ENV DB_PASSWORD=Sup3rS3cret!
$ trivy image --scanners secret shop-api:2.3
HIGH: secret in layer 4
```

Anyone who could pull the image could read the password. The team rotated the password, rebuilt the image without it, moved the value to the secrets manager with access limited to the service role, and added secret scanning to the build.

Common mistakes

- Secrets in code, images or logs
- One key for everything
- No rotation plan
- Giving everyone permission to decrypt

Check yourself

Where do secrets belong?

Answer: In a managed secrets store

What is envelope encryption?

Answer: A data key encrypts data and a master key encrypts the data key

Does encryption replace access control?

Answer: No, those with decrypt permission see the data

How should rotation work?

Answer: Create new, switch consumers, retire old

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 6. Network boundaries and private access

12 minutes. Limit exposure with private networks, security groups, private endpoints and sensible defaults for storage and databases.

Even perfect permissions are weaker when every resource is reachable from the internet. Network boundaries add a second layer: a service that cannot be reached cannot be attacked directly. Identity and network controls work together, and many cloud breaches began with a public storage bucket, an open database port or an unprotected management interface.

What you will learn

- Use private subnets and security groups sensibly
- Reach cloud services privately with endpoints
- Close the common public exposures
- Combine network and identity controls

Private by default

Place application servers and databases in private subnets without public addresses. Put only load balancers and gateways in public subnets. Outbound traffic goes through a NAT gateway or a proxy you can monitor. In Azure, the same idea uses virtual networks, subnets and network security groups.

Security groups and rules

A security group is a stateful firewall attached to a resource. Allow only the ports and sources needed, preferably referring to other security groups instead of wide address ranges. Be suspicious of `0.0.0.0/0` on anything other than a public web port, and never on SSH, RDP or database ports. Use session-based access (such as AWS Systems Manager Session Manager or Azure Bastion) instead of opening management ports.

Private endpoints

Cloud services such as object storage, key vaults and databases are normally reached through public addresses. **Private endpoints** (AWS PrivateLink and VPC endpoints, Azure Private Link) give the service a private address inside your network, so traffic never crosses the internet. Combine them with policies that deny access from anywhere else.

Public exposure checklist

Block public access on storage accounts and buckets at the account level; check for public snapshots and images; review public IP addresses regularly; protect web applications with a firewall and rate limits; and scan for open ports from outside. Remember that a network rule is not authorisation: a service that is reachable still has to check identity.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Network boundaries and private access fits together

Part	What it does	What goes wrong
Private by default	Place application servers and databases in private subnets without public addresses.	Public IPs on databases.
Security groups and rules	A security group is a stateful firewall attached to a resource.	Wide source ranges on management ports.
Private endpoints	Cloud services such as object storage, key vaults and databases are normally reached through public addresses.	Assuming a private network replaces authentication.
Public exposure checklist	Block public access on storage accounts and buckets at the account level; check for public snapshots and images; review public IP addresses regularly; protect web...	No account-level block on public storage.

Flow: Private by default to Security groups and rules / Security groups and rules to Private endpoints / Private endpoints to Public exposure checklist

Incident: The public bucket

Incident: *A storage bucket holding customer exports was reachable by anyone with the link.*

```
$ aws s3api get-public-access-block --bucket customer-exports
An error occurred (NoSuchPublicAccessBlockConfiguration)
$ aws s3api get-bucket-policy-status --bucket customer-exports
"IsPublic": true
```

The block on public access had never been enabled. The team turned it on for the whole account, replaced the sharing links with short-lived signed links, reviewed the access logs for outside reads, and added a rule that alerts on any public bucket.

Common mistakes

- Public IPs on databases
- Wide source ranges on management ports
- Assuming a private network replaces authentication
- No account-level block on public storage

Check yourself

Where should databases live?

Answer: In private subnets with narrow rules

What is a private endpoint?

Answer: A private address in your network for a cloud service

Which source is a red flag on SSH?

Answer: 0.0.0.0/0

Does a private network replace authentication?

Answer: No, services must still check identity

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 7. Logging, detection and audit trails

12 minutes. Record every control-plane action, protect the logs, and build alerts for the actions that signal trouble.

You cannot investigate what you did not record. Cloud providers log administrative actions: AWS CloudTrail, the Azure Activity Log, and Google Cloud Audit Logs. These logs tell you who did what, when, from where. Collected, protected and watched, they turn a breach from an unknown loss into a time-ordered story.

What you will learn

- Enable organisation-wide audit logging
- Read an audit record
- Protect logs from tampering
- Alert on high-risk actions

What to log

Management events (create, change, delete), data events for sensitive storage, sign-in logs from the identity provider, network flow logs, and application logs. Turn on audit logging in every account and region from day one, ideally centrally for the whole organisation.

Reading a record

An audit record names the **principal**, the **action**, the **resource**, the **source address**, the **time** and whether it succeeded.

```
{
  "eventTime": "2024-05-14T02:11:09Z",
  "eventName": "DeleteTrail",
  "userIdentity": { "type": "AssumedRole", "arn": "arn:aws:sts::123456789012:assumed-role/dev/anna \
    " },
  "sourceIPAddress": "203.0.113.45",
  "errorCode": null
}
```

Deleting a trail at 02:11 from an unfamiliar address is exactly the kind of event to alert on.

Protect the logs

Send logs to a separate account that few people can use. Make the storage append-only or write-once, enable integrity validation, and deny deletion. An attacker who can erase the log can hide the attack.

Detect and respond

Start with a short list of alerts: use of the root account, a change to logging, new access keys for privileged users, a policy that grants administrator rights, a public resource, sign-ins from unusual places, and many denied requests. Use managed threat detection services as a baseline. Write a short runbook for each alert: who looks, what they check, and when to disable the identity.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Logging, detection and audit trails fits together

Part	What it does	What goes wrong
What to log	Management events (create, change, delete), data events for sensitive storage, sign-in logs from the identity provider, network flow logs, and application logs.	Logs in the same account anyone can edit.
Reading a record	An audit record names the principal, the action, the resource, the source address, the time and whether it succeeded.	No alert on logging changes.
Protect the logs	Send logs to a separate account that few people can use.	Logging only in some regions.
Detect and respond	Start with a short list of alerts: use of the root account, a change to logging, new access keys for privileged users, a policy that grants administrator rights, a...	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: What to log to Reading a record / Reading a record to Protect the logs / Protect the logs to Detect and respond

Incident: Logging was off

Incident: *During an investigation the team found a gap of nine days with no audit records.*

```
$ aws cloudtrail get-trail-status --name org-trail
IsLogging: false
LatestDeliveryTime: 2024-05-05T01:02:13Z
```

A cost-cutting change had stopped the trail. The team re-enabled it, added an organisation policy preventing anyone from stopping it, set an alert on the stop event, and sent copies to a locked-down log account.

Common mistakes

- Logging only in some regions
- Logs in the same account anyone can edit
- No alert on logging changes
- Collecting logs nobody reads

Check yourself

What does an audit log show?

Answer: Who did what, when and from where

Where should logs be stored?

Answer: In a separate, protected account

Which event should always alert?

Answer: Audit logging being stopped

What does a runbook for an alert contain?

Answer: Who checks, what they check and when to disable access

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 8. Debugging access denied and cloud security incidents

14 minutes. Work out why a request was denied, fix it without over-granting, and respond in order to a suspected credential leak.

Two situations test everything in this skill. In the first, a legitimate request fails with "access denied" and you need the narrowest fix. In the second, a credential has leaked and you need to contain damage quickly. Both reward a calm, ordered method.

What you will learn

- Read an access-denied error
- Walk the evaluation chain
- Choose a narrow fix
- Respond to a leaked credential in order

Read the error

A good error names the principal, the action and the resource: "User arn:...:role/build is not authorized to perform s3:PutObject on arn:aws:s3:::reports-prod/x". That is already the answer to three questions. Check that the principal is the one you expect, because many failures are caused by running as a different role than intended.

Walk the chain

Check in order: is the right identity in use; does an identity policy allow the action on this resource; does a resource policy allow or deny it; does a boundary, organisation policy or deny statement restrict it; do conditions (network, MFA, tags, time) hold; and for data in storage or keys, does the key policy allow use. Policy simulators and the access analysis tools show which statement decided. Audit logs show the denied call with its error.

Fix narrowly

Add the specific action and resource, not a wildcard. If the cause is a missing condition such as the request not coming through the private endpoint, fix the path rather than removing the condition. Record why the access is needed.

Suspected credential leak

Act in this order: **contain** (deactivate the key or revoke the session, do not delete it yet); **assess** (audit logs for use of that credential, new users or keys, new resources, changed policies, copies of data); **eradicate** (remove attacker-created access, rotate related secrets); **recover** (replace with a role or short-lived credential); **learn** (how did it leak, what detection would have caught it sooner). Tell the right people early.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Debugging access denied and cloud security... fits together

Part	What it does	What goes wrong
Read the error	A good error names the principal, the action and the resource: "User arn:...:role/build is not authorized to perform s3:PutObject on arn:aws:s3:::reports-prod/x".	Check the Common mistakes list below this diagram for what usually goes wrong here.
Walk the chain	Check in order: is the right identity in use; does an identity policy allow the action on this resource; does a resource policy allow or deny it; does a boundary...	Fixing access errors with wildcards.
Fix narrowly	Add the specific action and resource, not a wildcard.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Suspected credential leak	Act in this order: contain (deactivate the key or revoke the session, do not delete it yet); assess (audit logs for use of that credential, new users or keys, new...	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Read the error to Walk the chain / Walk the chain to Fix narrowly / Fix narrowly to Suspected credential leak

Incident: Crypto servers in six regions

Incident: *A cloud bill spiked because someone launched large servers in regions the team never used.*

```
$ aws cloudtrail lookup-events --lookup-attributes AttributeKey=AccessKeyId,AttributeValue=AKIA... \
    EXAMPLE --max-results 3
RunInstances us-east-2  203.0.113.77
RunInstances ap-south-1 203.0.113.77
CreateAccessKey  203.0.113.77
```

The key had been exposed in a public repository. The team disabled it, terminated the servers, deleted the attacker-created key, checked for changed policies, enabled organisation guardrails that deny unused regions, and moved the application to roles.

Common mistakes

- Deleting evidence before capturing it
- Fixing access errors with wildcards
- Rotating one secret while others stay exposed
- No guardrail against unused regions

Check yourself

What does a good access error name?

Answer: Principal, action and resource

What is the first step for a leaked key?

Answer: Contain it by deactivating or revoking

Which fix is best for a missing permission?

Answer: Add the specific action on the specific resource

What shows which statement decided a request?

Answer: A policy simulator or access analysis

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 9. GCP IAM and service accounts

12 minutes. Understand Google Cloud IAM roles, the resource hierarchy, service accounts and how to avoid long-lived keys.

Google Cloud IAM has the same goals as other clouds but its own model: a resource hierarchy, role bindings and service accounts that are both identities and resources. Knowing it prevents common mistakes such as broad roles and exported keys.

What you will learn

- The resource hierarchy and inheritance
- Roles: basic, predefined and custom
- Service accounts and impersonation
- Avoiding keys with workload identity

Hierarchy

Policies are attached to **organization**, **folder**, **project** or individual resources, and grants are **inherited downward**. A role given at the folder applies to every project beneath it, so grant at the lowest level that works.

Roles

Basic roles such as Owner, Editor and Viewer are very broad and should be avoided in production. **Predefined roles** are scoped to a service, such as `roles/storage.objectViewer`. **Custom roles** collect exact permissions when no predefined role fits.

```
gcloud projects add-iam-policy-binding shop-prod \
  --member="serviceAccount:api@shop-prod.iam.gserviceaccount.com" \
  --role="roles/storage.objectViewer"
gcloud projects get-iam-policy shop-prod --format=json | jq '.bindings[].role' | sort | uniq -c
```

Service accounts

A workload runs **as** a service account. Create one per workload, give it minimal roles and attach it to the resource. To act as another account, use **impersonation**, which requires an explicit role.

```
gcloud iam service-accounts create api --display-name "API workload"
gcloud auth print-access-token --impersonate-service-account=api@shop-prod.iam.gserviceaccount.com
```

No keys

Downloaded JSON keys are long-lived secrets. Prefer service accounts attached to Compute, Cloud Run or GKE through **Workload Identity**, and **Workload Identity Federation** for workloads outside Google Cloud. Enforce an organization policy that disables key creation.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How GCP IAM and service accounts fits together

Part	What it does	What goes wrong
Hierarchy	Policies are attached to organization, folder, project or individual resources, and grants are inherited downward.	Granting roles at the folder when a project is enough.
Roles	Basic roles such as Owner, Editor and Viewer are very broad and should be avoided in production.	Using Owner or Editor in production.
Service accounts	A workload runs as a service account. Create one per workload, give it minimal roles and attach it to the resource.	Downloaded service account keys.
No keys	Downloaded JSON keys are long-lived secrets.	One service account shared by many workloads.

Flow: Hierarchy to Roles / Roles to Service accounts / Service accounts to No keys

Incident: The key in the repository

Incident: *A service account key was committed and abused.*

```
commit a1b2c3: add sa-key.json
audit: storage.objects.list from 203.0.113.50
principal: api@shop-prod.iam.gserviceaccount.com
```

The key never expired and was valid from anywhere. After revoking it the team disabled key creation by organization policy, moved workloads to workload identity and added a scanner that blocks key files in commits.

Common mistakes

- Using Owner or Editor in production
- Downloaded service account keys
- One service account shared by many workloads
- Granting roles at the folder when a project is enough

Check yourself

How do permissions flow in the hierarchy?

Answer: Downward from parent to child

Which roles should you avoid in production?

Answer: Basic roles like Owner and Editor

What replaces service account keys?

Answer: Workload identity

What does impersonation need?

Answer: An explicit role to act as the account

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 10. Workload identity federation and OIDC trust

12 minutes. Let CI systems and clusters call the cloud without stored keys by trusting signed tokens from an identity provider, and write tight trust conditions.

Long-lived access keys in CI systems are a top cause of cloud breaches. OpenID Connect federation removes them: the platform proves a job's identity and the cloud issues credentials that last minutes.

What you will learn

- How OIDC token exchange works
- Claims that matter: issuer, audience and subject
- Writing a narrow trust policy
- Debugging a failed exchange

The flow

1. The job requests an ID token from its platform, with an audience such as `sts.amazonaws.com`.
2. The token is a signed JWT with claims about the issuer, subject, repository, branch and more.
3. The cloud checks the signature with the issuer's public keys and compares the claims with the trust policy.
4. If they match, it returns temporary credentials.

A narrow trust policy

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {"Federated": "arn:aws:iam::123456789012:oidc-provider/token.actions.example.com" \
    },
    "Action": "sts:AssumeRoleWithWebIdentity",
    "Condition": {
      "StringEquals": {
        "token.actions.example.com:aud": "sts.amazonaws.com",
        "token.actions.example.com:sub": "repo:acme/shop:ref:refs/heads/main"
      }
    }
  }]
}
```

Always check **audience** and **subject**. Use exact `StringEquals` where you can. A wildcard on the subject, such as `repo:acme/*`, lets every repository assume the role.

Inspect and debug

```
# decode a token payload (never share tokens)
echo "$TOKEN" | cut -d. -f2 | base64 -d 2>/dev/null | jq '{iss,aud,sub}'
aws sts get-caller-identity
```

Common failures are a wrong audience, a subject that differs from the policy because of the environment or branch name, an issuer certificate change or a clock difference. Compare the token's claims with the policy line by line.

Operate

Give each pipeline a separate role with minimal permissions, log every assumed role session and alert on use from unexpected subjects.

MARGA by EknathaLabs
marga.eknathalabs.com

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Workload identity federation and OIDC trust fits together

Part	What it does	What goes wrong
The flow	The job requests an ID token from its platform, with an audience such as sts.amazonaws.com.	Wildcard subjects in trust policies.
A narrow trust policy	Always check audience and subject. Use exact StringEquals where you can.	Not checking the audience.
Inspect and debug	Common failures are a wrong audience, a subject that differs from the policy because of the environment or branch name, an issuer certificate change or a clock...	Check the Common mistakes list below this diagram for what usually goes wrong here.
Operate	Give each pipeline a separate role with minimal permissions, log every assumed role session and alert on use from unexpected subjects.	One role for all pipelines.

Flow: The flow to A narrow trust policy / A narrow trust policy to Inspect and debug / Inspect and debug to Operate

Incident: Any repository could deploy

Incident: *A trust policy used a wildcard subject.*

```
Condition: sub StringLike repo:acme/*
AssumeRoleWithWebIdentity by repo:acme/playground
role assumed: prod-deploy
```

A test repository could assume the production deploy role because the subject condition matched the whole organization. The team narrowed the condition to one repository and branch, split roles by environment and added alerts for new subjects.

Common mistakes

- Wildcard subjects in trust policies
- Not checking the audience
- One role for all pipelines
- Keeping a stored key as a fallback

Check yourself

What does the subject claim identify?

Answer: The workload, such as repo and branch

Why check audience?

Answer: To ensure the token was meant for this cloud

What is wrong with repo:acme/* as a condition?

Answer: Every repository matches

What do federated credentials last?

Answer: A short time

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 11. Permission boundaries, SCPs and guardrails

13 minutes. Set outer limits on what accounts and roles can ever do with SCPs, permission boundaries and organization policies, and understand how they combine with grants.

Giving teams freedom to create roles is fast, but it means an administrator in one account could grant anything. Guardrails set limits above that level so freedom stays inside safe borders.

What you will learn

- SCPs, permission boundaries and organization policies compared
- Effective permissions as an intersection
- Writing safe deny guardrails
- Testing before rollout

Three kinds of limit

- **Service control policy (SCP):** applies to all principals in the accounts it is attached to and never grants access, only limits it.
- **Permission boundary:** attached to a role or user to cap the permissions that identity can ever have, even if its policies say more.
- **Organization policy (Google Cloud) or Azure Policy:** enforce settings such as allowed regions or disabled keys.

The effective permission is what is **allowed by the identity policy and not blocked by** any boundary or SCP. An explicit deny always wins.

An example guardrail

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "ProtectAuditTrail",
    "Effect": "Deny",
    "Action": [
      "cloudtrail:StopLogging",
      "cloudtrail:DeleteTrail",
      "config:StopConfigurationRecorder"
    ],
    "Resource": "*"
  }, {
    "Sid": "OnlyApprovedRegions",
    "Effect": "Deny",
    "NotAction": ["iam:*", "organizations:*", "support:*"],
    "Resource": "*",
    "Condition": {"StringNotEquals": {"aws:RequestedRegion": ["eu-west-1", "eu-central-1"]}}
  }]
}
```

Test before enforcing

Apply to a test organizational unit first and check what would be blocked using access logs. Document break-glass exceptions with a time limit and approvals. Version guardrails in code and review them like application changes.

```
aws organizations list-policies --filter SERVICE_CONTROL_POLICY
aws iam simulate-principal-policy --policy-source-arn "$ROLE_ARN" --action-names cloudtrail:StopLo \
    gging
```

MARGA by EknathaLabs marga.eknathalabs.com

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Permission boundaries, SCPs and guardrails fits together

Part	What it does	What goes wrong
Three kinds of limit	Service control policy (SCP): applies to all principals in the accounts it is attached to and never grants access, only limits it.	Rolling out an SCP to all accounts at once.
An example guardrail	.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Test before enforcing	Apply to a test organizational unit first and check what would be blocked using access logs.	Many overlapping guardrails nobody understands.

Flow: Three kinds of limit to An example guardrail / An example guardrail to Test before enforcing

Incident: The deny that blocked everything

Incident: *A broad SCP denied actions that automation needed.*

```
AccessDenied: explicit deny in a service control policy
action: sts:AssumeRole
112 pipelines failed in 8 minutes
```

A region restriction was applied to the whole organization at once and forgot that global services use special regions. The team rolled it back, rewrote it with exceptions, tested on a pilot unit and added a change process for guardrails.

Common mistakes

- Rolling out an SCP to all accounts at once
- Forgetting global services in region rules
- Many overlapping guardrails nobody understands
- Permanent exceptions

Check yourself

Does an SCP grant permissions?

Answer: No, it only limits them

What wins between allow and deny?

Answer: Explicit deny

What does a permission boundary cap?

Answer: The maximum permissions of a role

How roll out a guardrail safely?

Answer: Pilot on a test unit first

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 12. IAM policy analysis and unused access

12 minutes. Find overly broad and unused permissions with access analyzers, last-used data and policy simulation, and remove them safely.

Writing least-privilege policy by guessing leads to either broken apps or broad wildcards. Cloud providers record what is used, so you can shrink access with evidence instead of fear.

What you will learn

- Finding unused and externally shared access
- Last-accessed information and generated policies
- Simulating and staging changes
- A regular review process

What the tools show

- **External access findings:** resources such as buckets or roles reachable from outside your organization.
- **Unused access findings:** roles, users and permissions with no use in a chosen period, such as 90 days.
- **Last accessed data:** when a service or action was last used by a principal.

```
aws accessanalyzer list-findings-v2 --analyzer-arn "$ANALYZER" \
  --filter '{"findingType": {"eq": ["UnusedPermission"]}}' | jq '.findings[].resource'
aws iam generate-service-last-accessed-details --arn "$ROLE_ARN"
```

Generate a tighter policy

Providers can analyze CloudTrail or audit logs for a role over a period and propose a policy limited to the actions seen. Review it for rare but needed actions, such as monthly jobs or failover, before applying.

Roll out in stages

1. Create the new, smaller policy beside the old one.
2. Use the **simulator** to check key actions.
3. Attach the new policy, remove the old one for a test role first and watch for denied calls.
4. Remove unneeded permissions in small steps with a rollback ready.

```
aws iam simulate-principal-policy --policy-source-arn "$ROLE_ARN" \
  --action-names s3:GetObject s3:PutObject --resource-arns "arn:aws:s3:::shop-data/*"
```

Review regularly

Run unused access reports monthly, assign each finding to an owner and track closure. Review external access findings immediately and document any intentional public resource.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How IAM policy analysis and unused access fits together

Part	What it does	What goes wrong
What the tools show	External access findings: resources such as buckets or roles reachable from outside your organization.	Ignoring external access findings.
Generate a tighter policy	Providers can analyze CloudTrail or audit logs for a role over a period and propose a policy limited to the actions seen.	Removing permissions without checking rare jobs.
Roll out in stages	Create the new, smaller policy beside the old one.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Review regularly	Run unused access reports monthly, assign each finding to an owner and track closure.	Applying generated policies without review.

Flow: What the tools show to Generate a tighter policy / Generate a tighter policy to Roll out in stages / Roll out in stages to Review regularly

Incident: Eleven permissions in use

Incident: *A role had hundreds of permissions and used almost none.*

```
role: ci-deploy
services allowed: 38
services used in 90 days: 4 (11 actions)
```

The role came from a template that granted broad access to get started. Nobody went back. Generating a policy from the audit logs and testing it in staging cut the role to what the pipeline needed, and the team scheduled a quarterly review.

Common mistakes

- Never reviewing roles after launch
- Removing permissions without checking rare jobs
- Ignoring external access findings
- Applying generated policies without review

Check yourself

What does last-accessed data show?

Answer: When permissions were last used

What is an external access finding?

Answer: A resource reachable from outside the organization

Why stage permission removal?

Answer: To catch denials before wide impact

What should you check in generated policies?

Answer: Rare but needed actions

MARGA by EknathaLabs
marga.eknathalabs.com

Cheat sheet: Cloud security and IAM

Policies, roles, scopes, keys, private access, audit logs and an incident checklist for AWS and Azure.

Read an AWS policy

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["s3:GetObject"],
    "Resource": "arn:aws:s3:::reports-prod/summaries/*"
  }]
}
```

Warning: *Action "" on Resource "" is administrator access. Never use it to make an error go away.*

Who am I and what can I do

```
aws sts get-caller-identity
aws iam list-attached-role-policies --role-name build
aws iam simulate-principal-policy --policy-source-arn ARN --action-names s3:PutObject --resource-a \
    rns ARN
aws iam generate-service-last-accessed-details --arn ARN
```

Warning: *Check which identity you are running as before changing any policy. Many denials come from the wrong role.*

Azure access

```
az account show
az role assignment list --assignee USER --all -o table
az role assignment create --assignee USER --role "Reader" --scope /subscriptions/SUB/resourceGroup \
    s/RG
az identity create --name app-identity --resource-group RG
```

Warning: *Assign roles at the lowest scope that works. A subscription-level Contributor can change everything below it.*

Secrets and keys

```
aws secretsmanager get-secret-value --secret-id prod/db --query SecretString --output text
aws kms describe-key --key-id alias/app-data
az keyvault secret show --vault-name kv-prod --name db-password --query value -o tsv
```

Warning: *Do not print secrets in logs or CI output. Fetch at run time, hold in memory, rotate with an overlap.*

Network exposure checks

```
aws s3api get-public-access-block --bucket NAME
aws ec2 describe-security-groups --filters Name=ip-permission.cidr,Values=0.0.0.0/0
az network nsg rule list --nsg-name NSG --resource-group RG -o table
```

Warning: *0.0.0.0/0 on SSH, RDP or a database port is an incident waiting to happen. Use session-based access instead.*

Audit trail lookups

```
aws cloudtrail lookup-events --lookup-attributes AttributeKey=EventName,AttributeValue=DeleteTrail \
     --max-results 5
aws cloudtrail get-trail-status --name org-trail
az monitor activity-log list --offset 1d -o table
```

Warning: *Alert on audit logging being stopped, root account use, new admin grants and public resources. Keep logs in a separate protected account.*

MARGA by EknathaLabs marga.eknathalabs.com

Leaked credential order

```
1. Contain  - deactivate the key or revoke the session (do not delete yet)
2. Assess   - audit log: what did this credential do, from where
3. Remove   - attacker-created users, keys, servers, policies
4. Rotate   - related secrets; replace with a role
5. Learn    - how it leaked; add detection
```

MARGA by EknathaLabs
marga.eknathalabs.com